

Exam 1: Regular Languages

Released: 12:30 pm Feb 19

This exam was released at 12:30 pm EST Thursday, February 19, and is due at **12:30 pm EST on Friday, February 20**. There are no late submissions; leave yourself enough time to upload your work.

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; we can't give points to an answer that we can't read.

Enter your work using the `answer` environment we have provided for you. The default color is `teal`, but you can choose to change it to any of `red`, `brown`, `magenta`, `violet`, `blue`, `darkgray`, or `black` like so:

```
\begin{answer}[<COLOR>]
  <YOUR WORK GOES HERE>
\end{answer}
```

This exam is open book and open notes. However, you may **NOT** use the internet to look up answers, and you may **NOT** discuss the exam with your fellow students or anyone else besides the TA's and the professor before the exam is due.

If you have questions during the exam, there will be a pinned thread on Piazza monitored by the TA's.

0. Honor Code 1 point

I commit to uphold the ideals of honor and integrity by refusing to betray the trust bestowed upon me as a member of the Georgia Tech community.

Signature: Aamod Varma

1. Short Answer 6 points

- (a) (2 points) Suppose you are trying to prove that $L = \{ww \mid w \in \Sigma^*\}$ is not regular using the pumping lemma. You have already assumed that L is regular, and let p be the pumping length. Explain why the string $1^p 1^p$ **cannot** be used as a choice for s that will lead to a contradiction.

Answer: In the pumping lemma we write a string $s = xyz$ so if we take $1^p 1^p = xyz$ then we need $|xy| \leq p$ and $|y| > 0$. This means we have xy a substring of 1^p . So let $x = 1^m$ and $y = 1^n$ where $m + n \leq p$ and $n > 0$. Now note that it's enough to show that there exists one decomposition in which all i keep the string $xy^i z$ in L . Now consider the case where $x = \varepsilon, y = 1^2, z = 1^{2p-2}$. Now note that in this case we have

$$xy^i z = 1^{2i} 1^{2p-2} = 1^{2(p+i-1)} = ww$$

where $w = 1^{p+i-1}$, in other words we have $xy^i z \in L$ for all i . Clearly this means that we cannot use this as a contradiction. In essence, there is a valid decomposition that does not produce a contradiction.

- (b) (2 points - *Section B only*) Give some example regular language L crafted from A and B and C such that $AB \cup C$ is L but A and B and C are each not regular. State what A , B , C and L represent in your specific example.

Answer:

- (c) (2 points - *Section X only*) Let L be a regular language, and let D be a DFA for L with q states. Show that one of the following must be true:

- L contains infinitely many strings

Answer:

Consider the case where we have a loop in the DFA D that is in an accepting path. That is there exists some state q_i for which there is a string s_0 that can traverse from the start state q_0 to q_i , a string s_1 that is not the empty string that loops from q_i to itself and a string s_2 that goes from q_i to the accept state. Clearly $s_0s_1s_2$ is a string accepted by this language. But now note that we can loop arbitrarily many times to create an infinite number of strings. More specifically any string $s_0s_1^is_2$ for $i \in \mathbb{N} \geq 0$ is a string accepted by our language. Now as all these strings for the varying i are distinct we have an infinite number of strings that are accepted by D .

- L contains only strings of length less than or equal to $q - 2$.

Answer: On the contrary assume there is no cycle in our DFA on any accepting path, this means that for any string that is accepted by our language, no state in that has a loop to itself. But this means that it's only possible to visit a state once in this path. Now note that if D has q states, then the maximum number of states that can be included in an accepting string is $q - 1$ states, this is because as each state is defined for a specific alphabet, that means even the accepting state needs to have an outgoing transition, if it transitions to any of the previous $q - 1$ states including itself then we have a cycle, so it must point to another state that cannot join back to the initial $q - 1$ states. Once we're in this sink state, we must not be able to become an accepting string. Hence we can have a maximum of $q - 1$ states in the accepting path. But $q - 1$ states have $q - 2$ edges or transitions. Hence the longest string we can have is $q - 2$.

2. Nondeterminism 6 points

Let $L^R = \{w^R | w \in L\}$, i.e. the set of strings which are the reverse of some string in L . Show by constructing an NFA that if L is a regular language, then L^R is also regular.

Answer: Let M be the DFA of L , we need to show that there exists a NFA that accepts L^R . Now we construct our NFA N by first copying over the DFA of L and modifying it as follows,

- First we reverse each transition. For any $\delta_M(q, \alpha) = p$ we have $\delta_N(p, \alpha) = q$.
- We define the accept state of our NFA as the start state of our DFA. So our accept state in N is just $\{q_0\}$ where q_0 is the start state of the DFA.

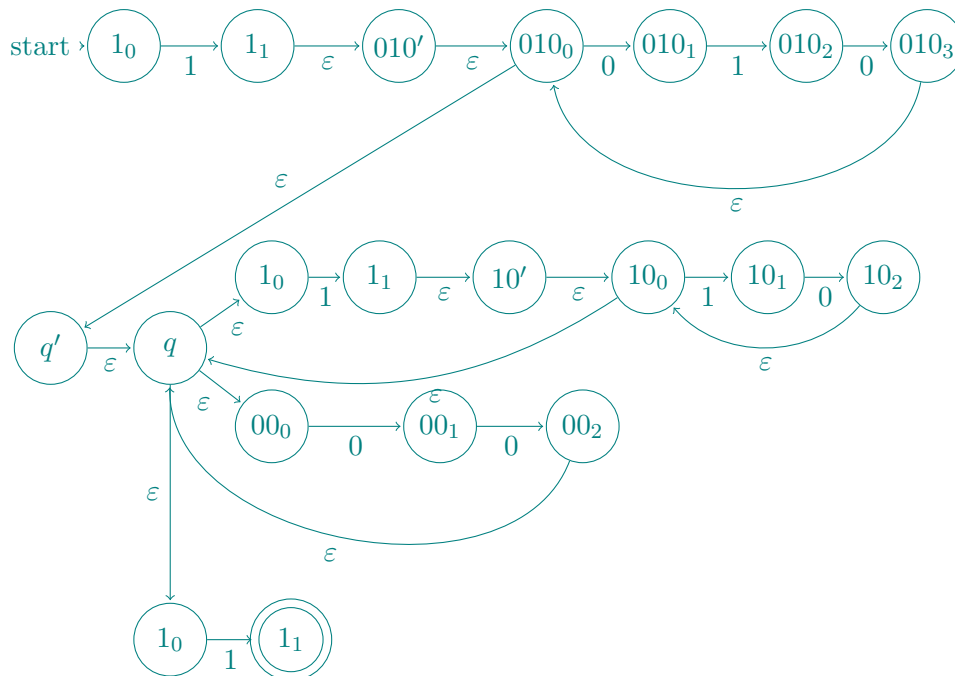
- (c) We define a new start state in N called q' and we add ε -transitions to all the accepts states in the previous DFA. i.e for all $q \in F$ we have $\delta_N(q', \varepsilon) = q$.

Now let s be a string in L represented as $s = x_1 \dots x_n$. Now clearly $x_1 \dots x_n$ is accepted by M . I.e we have a sequence of states $q_0, \dots, q_m$ such that q_0 is the start state q_m is the accept state and $\delta_M(q_i, x_j) = q_{i+1}$. Now note that in our NFA N we have q' as our start state that has an ε transition to each accept state of M which includes q_m . So if we consider the string $s' = x_n \dots x_1$ in our NFA N note that we have for every $\delta_M(q_i, x_j) = q_{i+1}$ that $\delta_N(q_{i+1}, x_j) = q_i$ hence we take the exact reverse direction until we reach $\delta_N(q_1, x_q) = q_0$ but note q_0 in N is defined as our accept state. Hence we have $x_m \dots x_1$ is accepted by our NFA. Similarly if N accepts a string s then reversing the accepting path gives a path in our DFA M which would accept w^R and hence we have $w \in L^R$.

3. Regular expressions 6 points

(3 points) Give an equivalent NFA for the following expression: $1(010)^*(1(10)^* \cup 00)^*1$

Answer:



(3 points)

Using an alphabet (Σ) of only lowercase vowels ('a', 'e', 'i', 'o', 'u'), give a regular expression representing all strings where 'a's cannot be the first or last letter.

Answer:

So we need to start with other letters and end with other letters except a . So for the first letter we have $(e \cup i \cup o \cup u)$, the middle can be any letter so we have $(a \cup e \cup i \cup o \cup u)^*$

and the last letter is non-a as well which is, $(e \cup i \cup o \cup u)$ so putting it together we have,

$$(e \cup i \cup o \cup u)(a \cup e \cup i \cup o \cup u)^*(e \cup i \cup o \cup u)$$

Now note that this implies our string is length 2 or above. So to include length 1 strings we have,

$$(e \cup i \cup o \cup u) \cup (e \cup i \cup o \cup u)(a \cup e \cup i \cup o \cup u)^*(e \cup i \cup o \cup u)$$

4. Pumping lemma 6 points

Use the pumping lemma to prove that $L = \{x\#y\#z \mid x \in \{0,1\}^*, y \in \{0,1\}^*, z \in \{0,1\}^*, \text{bin}(x) * \text{bin}(y) = \text{bin}(z)\}$ is not regular. Here, $\Sigma = \{0,1,\#\}$ and $\text{bin}(x)$ is the decimal value of x when interpreted as a binary string. For example: $\text{bin}(101) = 5$.

Answer: Let p be the pumping length, now consider the string $s = 10^p1\#1\#10^p1$. Note that we have $x = 10^p1, y = 1, z = 10^p1$. Clearly because $\text{bin}(y) = \text{bin}(1) = 1$ we trivially have $\text{bin}(x) * \text{bin}(y) = \text{bin}(x) = \text{bin}(z)$ as $x = z$. We also have $|s| \geq p$. Now if the language is regular we can write it as $s = uvw$ where $|uv| \leq p$ and $|v| > 0$ and for any $i \in \mathbb{N} \geq 0$ we have $uv^i w \in L$. Now this means we have $10^p1\#1\#10^p1 = uvw$ but note if $|uv| \leq p$ then $|uv|$ must be a prefix of 10^{p-1} (note that v cannot contain $\#$ as we can pump v in the beginning of the string which in this case is the first p letters and they don't contain $\#$, note that if it did contain $\#$ it's trivially a contradiction as we can pump it to any length and we now have more than 3 $\#$ which puts it outside the language). Now note that the possible decomposition we can have are,

- (a) v contains the leading 1 and hence is equal to 10^k for $1 \leq k \leq p-2$ and $u = \varepsilon$.
Now in this case simply consider $i = 0$. This gives us the string $uv^0w = w = 0^{p-k}1\#1\#10^p1$. Clearly $\text{bin}(0^k1) * \text{bin}(1) = \text{bin}(1) * \text{bin}(1) = 1 * 1 = 1 \neq \text{bin}(10^p1)$.
- (b) v does not have the leading 1 so it's just a bunch of zeroes so it is equal to 0^k for some $1 \leq k \leq p-1$. Consider again $i = 0$, now we have $10^{p-k}1\#1\#10^p1$. Now as $k \geq 1$ we cannot have $p-k = p$ so clearly the binary number $10^{p-k}1 \neq 10^p1$ and hence it is not true that $\text{bin}(10^{p-k}1) * \text{bin}(1) = \text{bin}(10^p1)$.